

AI Java Labs: Building Intelligent Java Applications

From HTTP fundamentals to advanced RAG systems →



Contact Info

Ken Kousen

Kousen IT, Inc.

- ken.kousen@kousenit.com
- <http://www.kousenit.com>
- <http://kousenit.org> (blog)
- Social Media:
 - [@kenkousen](#) (Twitter)
 - [@kousenit.com](#) (Bluesky)
 - <https://www.linkedin.com/in/kenkousen/> (LinkedIn)
- *Tales from the jar side* (free newsletter)
 - <https://kenkousen.substack.com>
 - <https://youtube.com/@talesfromthejarside>

What You'll Learn: Core Skills

- **HTTP Fundamentals:** Raw API integration patterns
- **LangChain4j Framework:** High-level AI abstractions
- **Text Generation:** OpenAI and Ollama models
- **Streaming Responses:** Real-time token-by-token output

What You'll Learn: Advanced Capabilities

- **Multimodal AI:** Vision, audio, and image generation
- **RAG Systems:** Document-based question answering
- **Cost Management:** Efficient testing and model selection
- **Modern Java:** Records, sealed interfaces, pattern matching

Repository Structure

```
1  AiJavaLabs/  
2  |— labs.md           # Progressive lab exercises  
3  |— slides.md         # This presentation  
4  |— build.gradle.kts  # Dependencies & test categories  
5  |— src/  
6      |— main/java/com/kousenit/  
7          |— demos/      # 9+ live demo classes  
8          |— *Service.java # Core implementations  
9          |— *Records.java # Data models  
10         |— test/        # Comprehensive test suite
```

- **9+ Live Demos:** Ready-to-run examples for each topic
- **Cost-Controlled Testing:** Free local tests, cheap API tests
- **Modern Java 21:** Records, sealed interfaces, pattern matching
- **Dual Learning Path:** Raw HTTP + LangChain4j frameworks

Demo Map: Core Demos

Demos 1-4

1. **QuickChatDemo** - Basic OpenAI chat
2. **LocalOllamaDemo** - Local AI models
3. **MultiModelDemo** - Compare providers
4. **TextToSpeechDemo** - Generate audio

Run Commands

```
1  # Pattern
2  ./gradlew run -PmainClass=com.kousenit.demos.DemoName
3
4  # Quick start
5  ./gradlew run -PmainClass=com.kousenit.demos.QuickChat
```

Prerequisites:

- `OPENAI_API_KEY` environment variable
- Ollama installed for local demos

Demo Map: Advanced Demos

Demos 7-11

7. **ToolCallingDemo** - Java methods as tools
8. **StructuredOutputDemo** - Records from model output
9. **StreamingDemo** - Real-time responses
10. **ApiComparisonDemo** / **ResponsesApiDemo**
11. **EasyRAGDemo** - Document Q&A

Key Concepts

- **Tool Calling:** Model-selected Java methods
- **Streaming:** Token-by-token output
- **API Comparison:** HTTP vs Framework
- **JSON Parsing:** Gson and Jackson navigation
- **RAG:** Document-powered AI

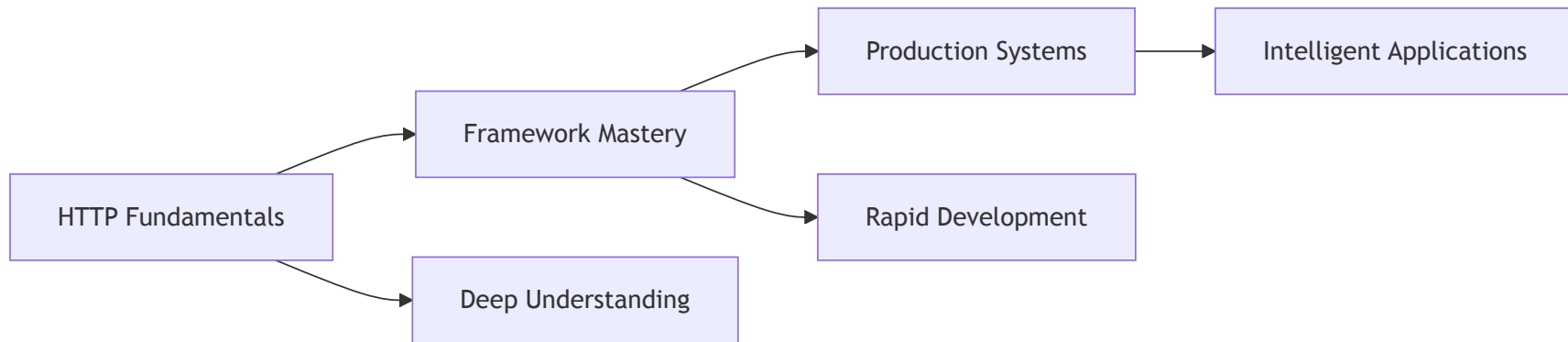
Course Philosophy: Understanding Both Levels

Why Learn Raw HTTP First?

- **True Understanding**: Know what frameworks do under the hood
- **Debugging Skills**: When frameworks fail, you can troubleshoot
- **Flexibility**: Not locked into any single framework
- **Performance**: Optimize for your specific needs

Then Master Frameworks

- **Productivity**: LangChain4j handles boilerplate
- **Best Practices**: Proven patterns and abstractions
- **Advanced Features**: RAG, streaming, memory
- **Rapid Development**: Focus on business logic



Prerequisites

Technical Requirements

- **Java 21+** (Records, sealed interfaces, pattern matching)
- **Gradle 8.4+** (Kotlin DSL, test categories)
- **LangChain4j 1.15.0** (current pinned framework)
- **Git** for repository management
- **Ollama** (optional, for local AI models)

Environment Setup

```
1  # Required API keys
2  export OPENAI_API_KEY=your_key
3  # Optional providers used in demos/tests
4  export GOOGLEAI_API_KEY=your_key
5  export PERPLEXITY_API_KEY=your_key
6
7  # Optional: Local AI models
8  curl -fsSL https://ollama.com/install.sh | sh
9  ollama pull gpt-oss
10 ollama pull moondream # For vision
11
12 # Clone and start
13 git clone <repo-url>
14 ./gradlew build
15 ./gradlew testDemo # Quick validation
16
17 # Slides
18 npm install
19 npx slidev slides.md
20 # Export to PDF (requires Playwright)
21 npx slidev export slides.md
```

Cost Management Strategy

Test Categories

- `@Tag("local")` - **Free** Ollama tests
- `@Tag("cheap")` - **Low-cost** nano models
- `@Tag("demo")` - **Fast** live demos
- `@Tag("expensive")` - **Avoided** by default

Gradle Tasks

```
1  ./gradlew testLocal      # $0
2  ./gradlew testCheap      # ~$0.0
3  ./gradlew testDemo       # ~$0.0
4  ./gradlew testNotExpensive
5  ./gradlew testOpenAI
```

Model Costs (Approximate; subject to change)

- **gpt-5-nano**: very low per-token cost
- **gpt-oss (local)**: Free with Ollama (OpenAI's open-source model)
- **gpt-image-2**: current GPT Image model
- **gpt-4o-mini-tts**: low cost per minute of audio

Smart Strategy: Develop with free local models, deploy with optimal cloud models

Demo 1-2: HTTP Fundamentals

Understanding Raw API Integration

Demo 1: Text-to-Speech with Raw HTTP

```
1 // Step 3: Complete implementation with file handling
2 public Path generateMp3(String model, String input, String voice) {
3     String payload = ""
4     {
5         "model": "%s",
6         "input": "%s",
7         "voice": "%s"
8     }
9     """".formatted(model, input.replaceAll("\\s+", " ").trim(), voice);
10
11     HttpRequest request = HttpRequest.newBuilder()
12         .uri(URI.create("https://api.openai.com/v1/audio/speech"))
13         .header("Authorization", "Bearer %s".formatted(OPENAI_API_KEY))
14         .header("Content-Type", "application/json")
15         .header("Accept", "audio/mpeg")
16         .POST(HttpRequest.BodyPublishers.ofString(payload))
17         .build();
18
19     try {
20         HttpResponse<Path> response =
21             client.send(request, HttpResponse.BodyHandlers.ofFile(getFilePath()));
22         return response.body();
23     } catch (IOException | InterruptedException e) {
24         throw new RuntimeException(e);
25     }
```

Demo 2: Model Discovery & JSON Parsing

```
1 // Service implementation with Gson parsing
2 public class OpenAiService {
3     private static final HttpClient client = HttpClient.newHttpClient();
4     private final Gson gson = new GsonBuilder()
5         .setFieldNamingPolicy(FieldNamingPolicy.LOWER_CASE_WITH_UNDERSCORES)
6         .create();
7
8     public ModelList listModels() throws IOException, InterruptedException {
9         HttpRequest request = HttpRequest.newBuilder()
10             .uri(URI.create("https://api.openai.com/v1/models"))
11             .header("Authorization", "Bearer " + System.getenv("OPENAI_API_KEY"))
12             .GET()
13             .build();
14
15         HttpResponse<String> response = client.send(request,
16             HttpResponse.BodyHandlers.ofString());
17         return gson.fromJson(response.body(), ModelList.class);
18     }
19 }
```

Key Learning: JSON parsing with records + Gson provides type-safe API responses

JSON Parsing: Gson vs Jackson

Gson Approach

```
1 // Using JsonElement tree navigation
2 JsonElement root = JsonParser.parseString(json);
3 JsonObject obj = root.getAsJsonObject();
4 JsonArray outputs = obj.getAsJsonArray("output");
5
6 for (JsonElement elem : outputs) {
7     JsonObject item = elem.getAsJsonObject();
8     if ("message".equals(item.get("type").getString())) {
9         // Extract nested content...
10     }
11 }
```

- Type-safe navigation
- Explicit type conversions
- Manual null checking needed

Jackson Approach

```
1 // Using JsonNode with at() method
2 ObjectMapper mapper = new ObjectMapper();
3 JsonNode root = mapper.readTree(json);
4
5 // Direct path with JSON Pointer!
6 String text = root.at("/output/0/content/0/text")
7                 .asText();
8
9 // Or safe navigation with path()
10 root.path("output")
11     .path(0)
12     .path("content")
13     .path(0)
14     .path("text");
```

- JSON Pointer navigation (`at()`)
- Safe chaining with `path()`
- No null pointer exceptions

JSON Pointer: Elegant Deep Navigation

RFC 6901 JSON Pointer Syntax

- **Direct path access:** `/output/0/content/0/text`
- **Array indexing:** Use numbers for array elements
- **Nested objects:** Chain with `/` separator

```
1 // Complex nested structure navigation - one line!
2 JsonNode root = mapper.readTree(response.body());
3
4 // Instead of multiple loops and null checks:
5 String text = root.at("/data/results/0/attributes/name").asText();
6
7 // With fallback:
8 String value = root.at("/missing/path").asText("default");
```

When to Use Which?

- **Gson:** Already in project, simpler API, Google ecosystem
- **Jackson:** Spring Boot default, JSON Pointer support, more features
- **Both:** Can coexist - use what fits your needs!

HTTP vs Framework Comparison

Raw HTTP Approach

```
1 // Manual request construction
2 HttpRequest request = HttpRequest.newBuilder()
3     .uri(URI.create(url))
4     .header("Authorization", "Bearer " + key)
5     .header("Content-Type", "application/json")
6     .POST(HttpRequest.BodyPublishers.ofString(
7         gson.toJson(payload)))
8     .build();
9
10 // Manual response handling
11 HttpResponse<String> response =
12     client.send(request, HttpResponse.BodyHandlers.ofString())
13 return gson.fromJson(response.body(), ResponseClass.class);
```

LangChain4j Framework

```
1 // High-level abstraction
2 ChatModel model = OpenAiChatModel.builder()
3     .apiKey(System.getenv("OPENAI_API_KEY"))
4     .modelName("gpt-5-nano")
5     .build();
6
7 // Simple method call
8 String response = model.chat("Hello, world!");
```

Both are valuable: Raw HTTP for understanding, frameworks for productivity

Demo 3-4: Local AI with Ollama

Privacy-First AI Integration

Demo 3: Ollama Setup and Architecture

Why Ollama?

- **Privacy**: No data leaves your machine
- **Cost**: Zero API costs after setup
- **Control**: Full model customization
- **Speed**: Local inference, no network delays



Setup Commands:

```
1  ollama pull gpt-oss      # Text generation (OpenAI open-source)
2  ollama pull moondream    # Vision analysis
3  ollama serve             # Start server (automatic on install)
```

Demo 4: Modern Java with Sealed Interfaces

```
1 // Step 3: Pattern matching with switch expressions (Java 21)
2 public OllamaResponse generate(OllamaRequest request) {
3     switch (request) {
4         case OllamaTextRequest textRequest -> {
5             logger.log(INFO, "Text request: {0}", textRequest.prompt());
6         }
7         case OllamaVisionRequest visionRequest -> {
8             logger.log(INFO, "Vision request with {0} images",
9                 visionRequest.images().size());
10        }
11        // Exhaustive - no default needed with sealed types!
12    }
13
14    // Same HTTP logic for both request types
15    return sendRequest(request);
16 }
```

Modern Java Benefits: Type safety, exhaustive pattern matching, compact constructors

Demo 5-6: Framework Integration

LangChain4j Power and Simplicity

Demo 5: Quick Chat with LangChain4j

```
1 // Step 2: Complete implementation - just one method call!
2 public class QuickChatDemo {
3     public static void main(String[] args) {
4         ChatModel model = OpenAiChatModel.builder()
5             .apiKey(System.getenv("OPENAI_API_KEY"))
6             .modelName("gpt-5-nano")
7             .build();
8
9         String prompt = "Explain what LangChain4j is in 2-3 sentences.";
10        System.out.println("Answer: " + model.chat(prompt));
11    }
12 }
```

Live Demo: Run `QuickChatDemo.java` - from complex HTTP to simple method call!

Demo 6: Multi-Model Provider Support

```
1 // Step 3: Or use local Ollama - still same interface!
2 ChatModel ollama = OllamaChatModel.builder()
3     .baseUrl("http://localhost:11434")
4     .modelName("gpt-oss")
5     .build();
6
7 String response = ollama.chat("What is the capital of France?");
8 System.out.println("Ollama: " + response);
```

Key Insight: Same `ChatModel` interface = vendor-agnostic code!

Provider Ecosystem

Cloud Providers

- **OpenAI** • gpt-5-nano, GPT Image
- **Perplexity** • Sonar (OpenAI-compatible API)
- **Google AI** • Gemini models
- **Anthropic** • Claude models
- **Azure OpenAI** • Enterprise GPT
- **AWS Bedrock** • Multiple models

Local Models

- **Ollama** • gpt-oss, llama3.1, mistral
- **Hugging Face** • Open source models
- **vLLM** • High-performance inference
- **LocalAI** • OpenAI-compatible API

Specialized Services

- **Vision** • moondream, LLaVA
- **Audio** • Whisper, TTS models
- **Embeddings** • sentence-transformers
- **Code** • CodeLlama, StarCoder

LangChain4j Advantage: Switch providers with just configuration changes!

Demo 7: Tool Calling

Let the Model Call Java Code

Tool Calling with LangChain4j

```
1 // Register tools with the AI service
2 TravelAssistant assistant = AiServices.builder(TravelAssistant.class)
3     .chatModel(model)
4     .tools(new TravelTools())
5     .build();
6
7 String answer = assistant.chat("""
8     I am driving from Hartford to Boston.
9     What is the weather like at both ends,
10    how far is it, and what will fuel cost?
11    """);
```

Key idea: The model chooses when to call a tool; Java still owns validation, side effects, and business rules.

Tool Calling Guardrails

- Keep tools small, named clearly, and easy to explain
- Validate all tool arguments before doing real work
- Avoid dangerous side effects in first-pass tools
- Log tool name, arguments, result, and failure separately
- Treat tool outputs as data, not as proof the model reasoned correctly

```
1  ./gradlew run -PmainClass=com.kousenit.demos.ToolCallingDemo
```

Demo 8: Structured Output Extraction

```
1 InquiryExtractor extractor = AIServices.builder(InquiryExtractor.class)
2     .chatModel(model)
3     .build();
4
5 CourseInquiry inquiry = extractor.extract(studentMessage);
6 validate(inquiry);
```

Key idea: Structured output turns model interpretation into Java data, but your application still validates the result.

```
1 ./gradlew run -PmainClass=com.kousenit.demos.StructuredOutputDemo
```

Demo 9-11: Streaming and Retrieval

Real-Time and Document-Powered AI

Demo 11: Easy RAG with LangChain4j

```
1 // Ask questions grounded in your documents
2 System.out.println(assistant.chat("Summarize Chapter 1."));
```

See `EasyRAGDemo.java` for a complete, runnable example.

Demo 9: Clean Streaming with Lambda Handlers

```
1 // With error handling - still clean and functional
2 var openai = OpenAiStreamingChatModel.builder()
3     .apiKey(System.getenv("OPENAI_API_KEY"))
4     .modelName("gpt-4o-mini")
5     .build();
6
7 // Handle both tokens and errors elegantly
8 openai.chat("Why is the sky blue?",
9     onPartialResponseAndError(
10         System.out::print, // Handle each token
11         error -> System.err.println("Error: " + error.getMessage())
12     ));
```

Live Demo: Run `StreamingDemo.java` to see tokens appear in real-time!

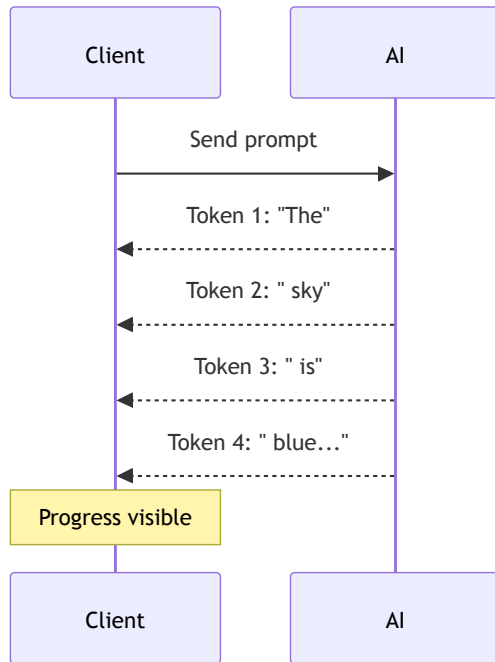
Streaming Architecture Benefits

User Experience

- **Immediate Feedback:** See progress instantly
- **Perceived Speed:** Feels faster
- **Interruptible:** Stop generation early
- **Engaging:** Watch AI "think"

Technical Benefits

- **Lower Latency:** First token quickly
- **Better Resources:** Process as arriving
- **Scalability:** Multiple streams
- **Error Recovery:** Graceful failures



Demo 11: Retrieval-Augmented Generation

Document-Powered AI

RAG: The Knowledge Problem

RAG Implementation with LangChain4j

```
1 // Step 4: Complete EasyRAGDemo implementation
2 public class EasyRAGDemo {
3     public static void main(String[] args) {
4         List<Document> documents = loadDocuments(toPath("documents/"), glob("*.txt"));
5
6         ChatModel chatModel = OpenAiChatModel.builder()
7             .apiKey(System.getenv("OPENAI_API_KEY"))
8             .modelName("gpt-5-nano")
9             .build();
10
11         Assistant assistant = AiServices.builder(Assistant.class)
12             .chatModel(chatModel)
13             .chatMemory(MessageWindowChatMemory.withMaxMessages(10))
14             .contentRetriever(createContentRetriever(documents))
15             .build();
16
17         // AI can now answer questions about your documents!
18         startConversationWith(assistant);
19     }
20 }
```

Live Demo: Run `EasyRAGDemo.java` - AI answers questions about YOUR documents!

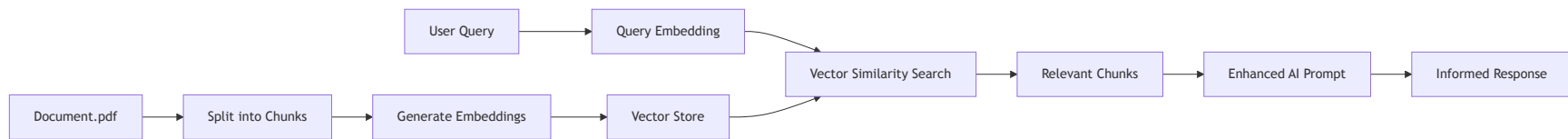
RAG Deep Dive: How It Works

Vector Embeddings

- **Numerical Representation**: Text → vectors (1536+ dimensions)
- **Semantic Similarity**: Similar meaning = closer vectors
- **Searchable**: Find relevant content by vector distance

Document Processing

- **Chunking**: Split documents into manageable pieces
- **Embedding**: Convert chunks to vectors
- **Storage**: Index in vector database
- **Retrieval**: Find most relevant chunks for query



Advanced Topics Overview

Production Considerations

- **Vector Databases:** Redis, Pinecone, Weaviate
- **Chunking Strategies:** Token-based, semantic, overlapping
- **Embedding Models:** OpenAI, sentence-transformers
- **Hybrid Search:** Vector + keyword search

Integration Patterns

- **Memory Management:** Conversation history
- **Multi-Modal RAG:** Text, images, audio
- **Real-Time Updates:** Dynamic document ingestion
- **Security:** Access control, data privacy

Developer Integration Patterns to Explore

Tool Calling

- Model-selected Java methods
- Application-owned validation and side effects
- Natural path to APIs, databases, and services
- Demo: `ToolCallingDemo`

Structured Outputs

- Ask for data contracts, not prose, when code consumes the result
- Map JSON output to records or DTOs
- Validate required fields and enum values
- Demo: `StructuredOutputDemo`

Why it matters: Most production AI work is not chat. It is model-assisted control flow around ordinary application code.

Testing Strategy

Smart Cost Control and Quality Assurance

Test Categories for Cost Control

Test Tags

```
1  @Test
2  @Tag("local") // Free
3  void testOllama() {
4      // $0 cost
5  }
6
7  @Test
8  @Tag("cheap") // Low cost
9  void testNano() {
10     // ~$0.001
11 }
12
13 @Test
14 @Tag("demo") // Fast
15 void quickDemo() {
16     // Optimized
17 }
```

Gradle Commands

```
1  # Free tests only
2  ./gradlew testLocal
3
4  # Low-cost tests
5  ./gradlew testCheap
6
7  # Demo tests
8  ./gradlew testDemo
9
10 # Exclude expensive
11 ./gradlew testNotExpensive
```

Strategy: Develop with `testLocal` , validate with `testCheap`

Advanced Test Selection

Provider-Specific Testing

```
1  # OpenAI API tests only
2  ./gradlew testOpenAI
3
4  # All tests (including expensive)
5  ./gradlew test
6
7  # Custom test selection
8  ./gradlew test --tests "*OpenAi*"
```

Environment Guards

```
1  @Test
2  @EnabledIfEnvironmentVariable(
3      named = "OPENAI_API_KEY",
4      matches = ".+")
5  void testRequiresApiKey() {
6      // Only runs if key is set
7  }
```

Smart Development: Use free local models for TDD, validate with cloud models before deployment

Raw HTTP: Timeouts & Retries (Advanced)

- Add timeouts to `HttpClient` requests to avoid hanging calls
- Implement simple retry with jitter for transient errors (429, 5xx)
- Log request IDs and status to aid debugging
- Prefer idempotency where possible for safe retries

```
1  HttpClient client = HttpClient.newBuilder()
2      .connectTimeout(Duration.ofSeconds(10))
3      .build();
4
5  HttpRequest request = HttpRequest.newBuilder(uri)
6      .timeout(Duration.ofSeconds(30))
7      .build();
```

Testing Patterns for AI Applications

Unit Testing Approaches

```
1  @Test
2  void shouldParseModelList() {
3      String json = ""
4          {"data": [
5              {"id": "gpt-4", "created": 123}
6          ]}
7      "";
8      ModelList result = gson.fromJson(json, ModelList.class);
9      assertThat(result.data()).hasSize(1);
10 }
11
12 @Test
13 void shouldHandleApiError() {
14     // Test error scenarios
15     assertThrows(RuntimeException.class,
16         () -> service.generateWithInvalidKey());
17 }
```

Integration Testing

```
1  @Test
2  @Tag("local")
3  void shouldStreamTokens() {
4      List<String> tokens = new ArrayList<>();
5
6      ollamaService.generateStreaming("gpt-oss",
7          "Count to 5",
8          token -> tokens.add(token)
9      );
10
11      assertThat(tokens).isNotEmpty();
12      assertThat(String.join("", tokens))
13          .containsPattern("1.*2.*3.*4.*5");
14 }
```

Quality Assurance for Non-Deterministic Systems

- **Content Assertions:** Check for key concepts, not exact strings
- **Structural Validation:** Verify JSON format, required fields
- **Error Handling:** Test timeout, rate limits, invalid input
- **Performance Bounds:** Response time, token limits
- **Cost Monitoring:** Track API usage in tests

```
1  @Test
2  @Tag("cheap")
3  void shouldAnswerSkyColorQuestion() {
4      String response = chatModel.chat("Why is the sky blue?");
5
6      // Flexible content assertion
7      assertThat(response.toLowerCase())
8          .containsAnyOf("scattering", "wavelength", "atmosphere", "light");
9
10     // Structural assertion
11     assertThat(response).hasSizeGreaterThan(10);
12     assertThat(response).doesNotContain("I don't know");
13 }
```

Course Summary

From HTTP to Intelligent Applications

Your AI Java Journey

Foundation Built

- **HTTP Fundamentals**: Raw API integration patterns
- **Modern Java**: Records, sealed interfaces, pattern matching
- **Framework Mastery**: LangChain4j abstractions
- **Cost Management**: Smart testing strategies

Result: You can build intelligent Java applications with confidence! 🚀

Advanced Capabilities

- **Multi-Modal AI**: Text, vision, audio integration
- **Streaming Responses**: Real-time user experiences
- **RAG Systems**: Document-powered AI
- **Production Patterns**: Error handling, structured outputs, monitoring

Demo Classes Mastered

Core Demos

- [QuickChatDemo](#) - Fast OpenAI integration
- [TextToSpeechDemo](#) - Audio generation
- [LocalOllamaDemo](#) - Privacy-first AI
- [MultiModelDemo](#) - Provider comparison

Advanced Demos

- [StreamingDemo](#) - Real-time responses
- [ResponsesApiDemo](#) - Raw HTTP patterns
- [ApiComparisonDemo](#) - Framework vs raw
- [ToolCallingDemo](#) - Java methods as tools
- [StructuredOutputDemo](#) - Records from model output
- [EasyRAGDemo](#) - Document Q&A

All Ready for Live Demonstration: Each demo is optimized for training delivery

Key Takeaways: Foundation

1. **Understand Both Levels**: Raw HTTP gives you debugging superpowers, frameworks give you productivity
2. **Modern Java Shines**: Records, sealed interfaces, and pattern matching are perfect for AI APIs
3. **LangChain4j is Powerful**: Unified API across all providers enables true portability

Key Takeaways: Best Practices

4. **Cost Management Matters**: Use test categories and local models for development
5. **RAG Needs Guardrails**: Give AI access to your documents and verify what it retrieves
6. **Streaming Improves UX**: Real-time responses feel much faster to users
7. **Contracts Matter**: Use tool calling and structured outputs when code consumes model results
8. **Test Smart, Not Hard**: Use local models for TDD, cloud models for validation

Next Steps: Continue Your AI Journey

Hands-On Practice

- **Explore the repository:** Try the demo classes
- **Complete the labs:** Work through the progressive exercises
- **Build custom integrations:** Add your own AI providers
- **Production deployment:** Scale with real applications

Resources:

- [LangChain4j Documentation](#)
- [Course Repository](#)
- [Ollama Models](#)

Advanced Topics

- **Vector databases:** Redis, Pinecone, Weaviate
- **Multi-agent systems:** Coordinated AI workflows
- **Function calling:** Give AI controlled access to your APIs
- **Structured outputs:** Turn model responses into validated Java data
- **Custom embeddings:** Domain-specific models

Production Considerations

Operational Excellence

- **API Cost Monitoring**: Track usage across models
- **Rate Limiting**: Handle quotas gracefully
- **Error Recovery**: Circuit breakers, retries
- **Security**: API key management, input validation

Performance Optimization

- **Model Selection**: Right model for the task
- **Caching Strategies**: Cache embeddings and responses
- **Batch Processing**: Optimize API usage
- **Local Models**: Privacy and cost benefits

Thank You!

Questions?

Kenneth Kousen *Java Champion, Author, Speaker*

kousenit.com | [@kenkousen](https://twitter.com/kenkousen)



Ready to build intelligent Java applications!

Repository: github.com/kousen/AiJavaLabs